

Pluma

Banco de dados

Armazenamento atual no navegador campo a campo, acervo de textos, palavras e músicas, e a arquitetura sugerida em PostgreSQL para contas, sincronização e ranking por música: diagrama de relacionamentos, tabelas, integridade, índices, consultas prontas, migração do histórico e segurança.

Copyright (©) 2026 Erik Pimenta
Versão online: https://ravelroot.github.io/Typing-Practice-Platform/

Versão 1.1.0 · Novembro de 2026 · Todos os direitos reservados

Versão 1.1.0 · Novembro de 2026

Novidades da versão 1.1.0

O modo **Ritmo** acrescenta dados novos no navegador e no acervo, e a arquitetura sugerida ganha as tabelas `track` e `rhythm_result`. Nada muda nos dados que já existiam: os resultados antigos continuam válidos e legíveis.

> **Onde está:** os campos novos aparecem marcados com NOVO nas tabelas dos capítulos "Armazenamento atual" e "Tabelas".

Onde	O que muda
`pluma:prefs`	Modo `rhythm` e os campos `musicVolume`, `rhythmOffset` e `rhythmTrack`
`pluma:historico`	Resultados com `mode = "rhythm"` e os campos opcionais `score`, `grade`, `perfect`, `great`, `good`, `miss`, `title`, `bpm` e `track`
Acervo	Arquivo novo `web/data/musicas.json`, com as 56 faixas
CSV	Oito colunas novas: pontos, nota, perfeitos, ótimos, bons, perdidas, música e BPM; células que começariam uma fórmula ganham um apóstrofo
Arquitetura sugerida	Tabelas `track` (catálogo) e `rhythm_result` (detalhe da partida), visão `rhythm_record` e índice de ranking por música

Sumário

Visão geral 4
- Quando adotar um banco 4
- Princípios 4

Armazenamento atual 5
- Chaves do localStorage 5
- Preferências (pluma:prefs) 5
- Resultado (pluma:historico) 6
- Acervo (web/data) 6

Diagrama de relacionamentos 7

Regras e convenções 8
- Integridade 8
- Exclusões em cascata 8

Tabelas 9
- app_user 9
- user_preference 9
- result 9
- rhythm_result [NOVO] 10
- missed_key 10
- track [NOVO] 11
- phrase e word 11

Índices e consultas prontas 12
- Recordes por categoria 12
- Ranking de uma música [NOVO] 12
- Evolução dos últimos 30 testes 12
- Letras que mais passam no Ritmo [NOVO] 12

Migração do navegador para o banco 13

Segurança e privacidade 14

Preparar o PostgreSQL 15

Visão geral

O Pluma não tem banco de dados hoje, de propósito: o site é um arquivo só, abre sem servidor e não envia nada para a internet. Os dados de cada pessoa ficam no `localStorage` do navegador e o acervo (textos, palavras e músicas) vai embutido no `Pluma.html` no build.

Este documento tem duas partes: o **armazenamento atual**, campo a campo, e a **arquitetura sugerida** para quando o Pluma precisar de contas, sincronizar o histórico entre aparelhos ou ter ranking por música. A proposta usa PostgreSQL 16 e foi desenhada para receber o histórico que já existe nos navegadores sem perder nada.

Quando adotar um banco

Necessidade	Sem banco (hoje)	Com o banco sugerido
Histórico em vários aparelhos	Cada navegador tem o seu; dá para exportar CSV	Um histórico por conta, sincronizado
Ranking por música	Só os recordes da própria pessoa	Ranking entre pessoas, por música e categoria
Turmas e escolas	Não há	Grupos, metas e relatórios por turma
Privacidade	Nada sai do navegador	Dados mínimos, por conta, com exclusão em cascata

Princípios

- O navegador continua funcionando sozinho: o banco é opcional e só recebe o que a pessoa sincronizar.
- Os números ficam inteiros, como no código (`accuracy_bp` em pontos-base, `net_per_min_x100` em centésimos), para que o banco e a tela mostrem exatamente o mesmo valor.
- Cada resultado do navegador tem um `id` próprio; o banco guarda esse `client_id` e recusa duplicatas, então sincronizar duas vezes não duplica nada.
- O acervo é a fonte da verdade no repositório; o banco só espelha o catálogo para as chaves estrangeiras e o ranking.
- Texto customizado e documentos enviados nunca vão para o banco.

Armazenamento atual

Chaves do localStorage

Chave	Conteúdo	Limite
`pluma:prefs`	Preferências (objeto JSON)	Texto customizado até 120 mil caracteres
`pluma:historico`	Lista de resultados, do mais novo ao mais antigo	500 resultados; os mais antigos saem

Tudo o que é lido do `localStorage` é validado: valores fora da lista voltam ao padrão, números são limitados ao intervalo e resultados sem `id`, `at` ou `netPerMinX100` são descartados.

Preferências (pluma:prefs)

Campo	Tipo	Padrão	Validação
`mode`	texto	`test`	`free`, `test` ou `rhythm` NOVO
`kind`	texto	`text`	um dos nove conteúdos
`size`	texto	`short`	`short`, `medium` ou `long`
`complexity`	texto	`easy`	`easy`, `medium` ou `hard`
`theme`	texto ou nulo	nulo (segue o sistema)	`light` ou `dark`
`palette`	texto	`pluma`	`pluma`, `bento`, `sonda` ou `nexo`
`sound`, `keyboard`, `effects`, `stopOnError`, `tutorialDone`	booleano	—	booleano
`caret`	texto	`line`	`line`, `block` ou `underline`
`fontSize`	texto	`medium`	`small`, `medium` ou `large`
`customText`, `customName`, `customPart`	texto, texto, inteiro	vazio, nulo, 0	até 120 mil e 200 caracteres; parte ≥ 0
`musicVolume` NOVO	inteiro	70	0 a 100
`rhythmOffset` NOVO	inteiro (ms)	0	−200 a 200
`rhythmTrack` NOVO	texto	vazio (primeira faixa)	`^[a-z0-9-]{0,60}$`; faixa inexistente vira a primeira

Resultado (pluma:historico)

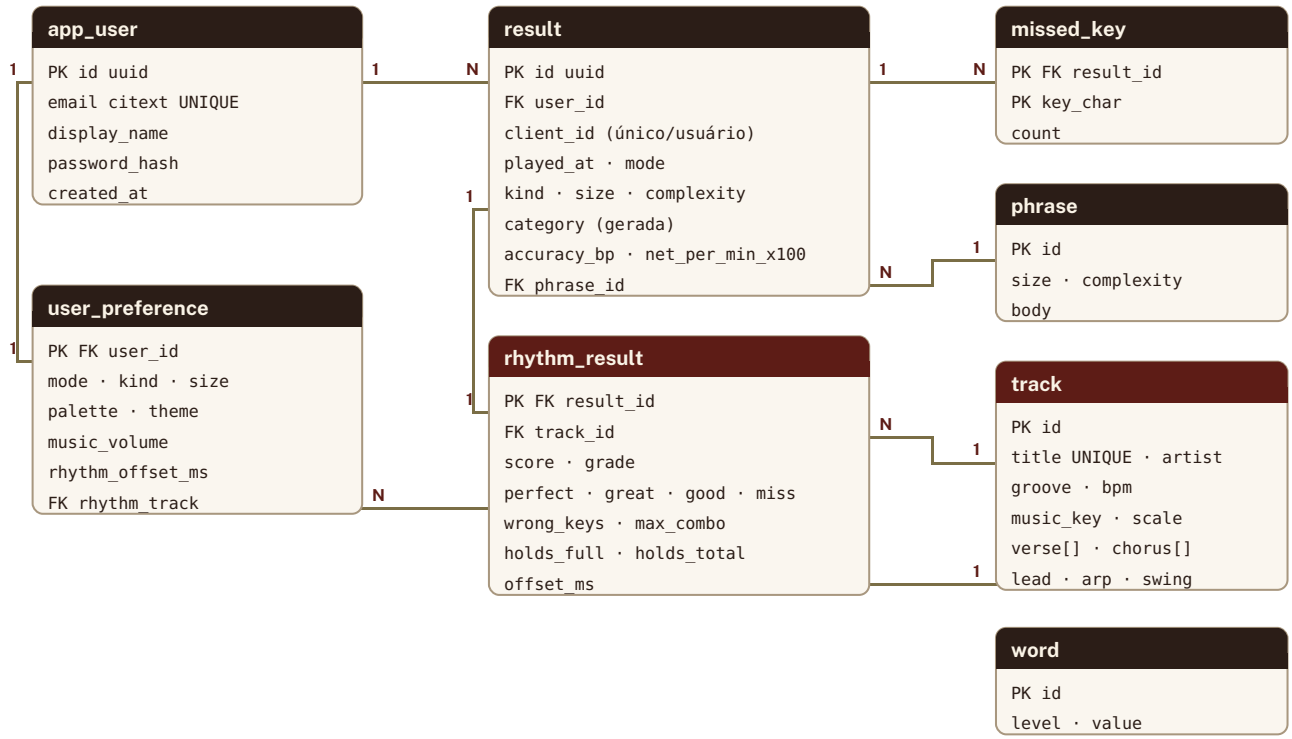
Campo	Tipo	Significado
`id`	texto	Identificador gerado no navegador (tempo em base 36 + 5 caracteres aleatórios)
`at`	texto ISO 8601	Data e hora do resultado
`mode`, `kind`, `size`, `complexity`	texto	Categoria
`seed`, `phraseId`	inteiro, texto	Semente do texto ou da partitura e frase do acervo
`texts`	inteiro	Textos da sessão (Livre) ou 1
`chars`	inteiro	Caracteres do texto; no Ritmo, notas da música
`durationMs`	inteiro	Tempo final, ativo ou duração da música
`keystrokes`, `correct`	inteiro	Toques e acertos; no Ritmo, teclas apertadas e notas acertadas
`errors`, `uncorrected`	inteiro	Erros e não corrigidos; no Ritmo, perdidas + teclas erradas e perdidas
`accuracyBp`	inteiro	Precisão em pontos-base (10000 = 100%)
`net`, `netPerMinX100`, `wpmX100`	inteiro	Toques líquidos e por minuto (×100); no Ritmo, letras acertadas por minuto
`consistencyBp`, `maxStreak`	inteiro ou nulo, inteiro	Consistência e maior sequência (no Ritmo, maior combo)
`score`, `grade` NOVO	inteiro, texto	Pontos e nota de S a D
`perfect`, `great`, `good`, `miss` NOVO	inteiro	Contagem de cada julgamento
`title`, `bpm`, `track` NOVO	texto, inteiro, texto	Música, andamento e id da faixa

Acervo (web/data)

Arquivo	Formato
`textos.json`	`{ version, phrases: [{ id, size, complexity, text }] }` — 204 frases
`palavras.json`	`{ version, easy[], medium[], hard[], sounds[] }` — 592 palavras e 73 onomatopeias
`musicas.json` NOVO	`{ version, tracks: [{ id, title, artist, groove, bpm, key, scale, verse[], chorus[], lead, arp, swing }] }` — 56 faixas

Diagrama de relacionamentos

A arquitetura sugerida tem oito tabelas e duas visões. As linhas mostram as chaves estrangeiras; 1 e N indicam a cardinalidade.



Área	Tabelas
Contas	`app_user`, `user_preference`
Resultados	`result`, `rhythm_result` NOVO, `missed_key`
Acervo	`phrase`, `word`, `track` NOVO
Visões	`typing_record`, `rhythm_record` NOVO

Regras e convenções

- Nomes de tabelas e colunas em inglês e `snake_case`, iguais aos campos do código (`netPerMinX100` vira `net_per_min_x100`).
- Listas fechadas como `varchar` com `CHECK`, e não `ENUM`, para que a coluna gerada `category` seja imutável e para facilitar acrescentar valores com uma migração simples.
- Datas em `timestamptz`, sempre em UTC; a tela converte para o fuso de quem vê.
- Contagens com `CHECK (… >= 0)`; precisão e consistência entre 0 e 10000.
- Toda tabela de dados pessoais tem `user_id` e Row Level Security.

Integridade

Restrição	Tabela	Regra
`result_client_unique`	result	Um `client_id` por pessoa: sincronizar de novo não duplica
`result_correct_valid`	result	`correct <= keystrokes`
`result_uncorrected_valid`	result	`uncorrected <= errors`
`rhythm_notes_sum` NOVO	rhythm_result	`perfect + great + good + miss = notes`
`rhythm_holds_valid` NOVO	rhythm_result	`holds_full <= holds_total`
`rhythm_grade_valid` NOVO	rhythm_result	`grade IN (S, A, B, C, D)`
`track_id_format` NOVO	track	`id ~ ^[a-z0-9-]{2,60}$`
`track_degrees_valid` NOVO	track	de 1 a 16 graus entre 0 e 6 no verso e no refrão
`track_swing_valid` NOVO	track	`swing` entre 0 e 0,4

Exclusões em cascata

Excluir uma conta apaga as preferências, os resultados, os detalhes do Ritmo e as letras perdidas na mesma transação. Faixas do catálogo nunca são apagadas: uma faixa retirada fica com `active = false`, para não quebrar os resultados antigos que apontam para ela.

Tabelas

app_user

Campo	Tipo	Regras
`id`	uuid	PK, `gen_random_uuid()`
`email`	citext	único, obrigatório
`display_name`	varchar(60)	nome mostrado no ranking
`password_hash`	text	Argon2id ou bcrypt; nunca a senha
`created_at`, `deleted_at`	timestamptz	criação e exclusão lógica antes da exclusão definitiva

user_preference

Campo	Tipo	Regras
`user_id`	uuid	PK e FK para `app_user`, cascata
`mode`, `kind`, `size`, `complexity`	varchar	mesmas listas do navegador
`theme`, `palette`, `caret`, `font_size`	varchar	mesmas listas do navegador
`sound`, `keyboard`, `effects`, `stop_on_error`	boolean	—
`music_volume` NOVO	smallint	0 a 100, padrão 70
`rhythm_offset_ms` NOVO	smallint	−200 a 200, padrão 0
`rhythm_track` NOVO	varchar(60)	FK para `track`, nulo se a faixa sair
`updated_at`	timestamptz	a versão mais nova vence na sincronização

result

Campo	Tipo	Regras
`id`	uuid	PK
`user_id`	uuid	FK para `app_user`, cascata
`client_id`	varchar(24)	`id` do navegador; único por pessoa
`played_at`	timestamptz	`at` do navegador

Campo	Tipo	Regras
`mode`, `kind`, `size`, `complexity`	varchar	categoria
`category`	text gerada	`custom` ou `kind/size/complexity`
`seed`, `phrase_id`	bigint, varchar(20)	semente (0 a 4294967295) e FK opcional para `phrase`
`texts`, `chars`, `duration_ms`	integer	≥ 1, ≥ 0, ≥ 0
`keystrokes`, `correct`, `errors`, `uncorrected`	integer	≥ 0, com as regras de integridade
`accuracy_bp`, `consistency_bp`	integer	0 a 10000 (consistência pode ser nula)
`net`, `net_per_min_x100`, `wpm_x100`, `max_streak`	integer	≥ 0

rhythm_result [NOVO]

Campo	Tipo	Regras
`result_id`	uuid	PK e FK para `result`, cascata
`track_id`	varchar(60)	FK para `track`
`bpm`	smallint	60 a 200
`score`	integer	≥ 0
`grade`	char(1)	S, A, B, C ou D
`perfect`, `great`, `good`, `miss`, `notes`	integer	soma dos quatro = `notes`
`wrong_keys`, `max_combo`	integer	≥ 0, padrão 0
`holds_full`, `holds_total`	integer	`holds_full <= holds_total`, padrão 0
`offset_ms`	smallint	média da sincronia, −140 a 140, pode ser nula

missed_key

Campo	Tipo	Regras
`result_id`	uuid	PK e FK para `result`, cascata
`key_char`	varchar(4)	PK; a tecla (ou letra do Ritmo)
`count`	integer	> 0

track [NOVO]

Campo	Tipo	Regras
`id`	varchar(60)	PK, formato do `musicas.json`
`title`, `artist`	varchar(80), varchar(60)	título único
`groove`	varchar(12)	um dos 18 estilos
`bpm`	smallint	60 a 200
`music_key`, `scale`	varchar	tom e escala
`verse`, `chorus`	smallint[]	1 a 16 graus entre 0 e 6
`lead`, `arp`, `swing`	varchar, varchar, numeric(3,2)	listas fechadas; swing 0 a 0,4
`active`	boolean	padrão true

phrase e word

Campo	Tipo	Regras
`phrase.id`	varchar(20)	PK, o `id` do `textos.json`
`phrase.size`, `phrase.complexity`, `phrase.body`	varchar, varchar, text	10 a 600 caracteres, único
`word.id`	serial	PK
`word.level`, `word.value`	varchar(6), varchar(40)	`easy`, `medium`, `hard` ou `sounds`; único por nível

Índices e consultas prontas

Índice	Para quê
`result (user_id, played_at DESC)`	Últimos resultados e o painel de histórico
`result (user_id, mode, category, net_per_min_x100 DESC)`	Recordes dos testes por categoria
`result (user_id, played_at DESC) WHERE mode = 'test'`	Evolução dos últimos testes
`rhythm_result (track_id, score DESC)` NOVO	Ranking de cada música
`missed_key (key_char)`	Letras que mais passam no geral

Recordes por categoria

```
SELECT category, net_per_min_x100, accuracy_bp, played_at
FROM typing_record
WHERE user_id = $1
ORDER BY net_per_min_x100 DESC;
```

Ranking de uma música [NOVO]

```
SELECT u.display_name, rr.score, rr.grade, r.accuracy_bp, r.played_at
FROM rhythm_record rr
JOIN result r ON r.id = rr.result_id
JOIN app_user u ON u.id = r.user_id AND u.deleted_at IS NULL
WHERE rr.track_id = $1 AND r.category = $2
ORDER BY rr.score DESC, r.played_at
LIMIT 10;
```

Evolução dos últimos 30 testes

```
SELECT played_at, net_per_min_x100 / 100.0 AS tlm
FROM result
WHERE user_id = $1 AND mode = 'test'
ORDER BY played_at DESC
LIMIT 30;
```

Letras que mais passam no Ritmo [NOVO]

```
SELECT mk.key_char, sum(mk.count) AS perdidas
FROM missed_key mk
JOIN result r ON r.id = mk.result_id
WHERE r.user_id = $1 AND r.mode = 'rhythm'
GROUP BY mk.key_char
ORDER BY perdidas DESC
LIMIT 6;
```

Migração do navegador para o banco

A sincronização manda o conteúdo de `pluma:historico` para a API, que valida cada resultado com as mesmas regras do navegador e grava tudo numa transação. Como o `client_id` é único por pessoa, mandar o mesmo histórico de novo não duplica nada; resultados novos entram e os já existentes são ignorados.

```
WITH data AS (
  SELECT * FROM jsonb_to_recordset($1::jsonb) AS e(
    id text, at timestamptz, mode text, kind text, size text, complexity text,
    seed bigint, "phraseId" text, texts int, chars int, "durationMs" int,
    keystrokes int, correct int, errors int, uncorrected int, "accuracyBp" int,
    net int, "netPerMinX100" int, "wpmX100" int, "consistencyBp" int, "maxStreak" int,
    score int, grade text, perfect int, great int, good int, miss int, track text, bpm int)
), ins AS (
  INSERT INTO result (user_id, client_id, played_at, mode, kind, size, complexity, seed,
    phrase_id, texts, chars, duration_ms, keystrokes, correct, errors, uncorrected,
    accuracy_bp, net, net_per_min_x100, wpm_x100, consistency_bp, max_streak)
  SELECT $2, id, at, mode, kind, size, complexity, seed, "phraseId", texts, chars,
    "durationMs", keystrokes, correct, errors, uncorrected, "accuracyBp", net,
    "netPerMinX100", "wpmX100", "consistencyBp", "maxStreak"
  FROM data
  ON CONFLICT (user_id, client_id) DO NOTHING
  RETURNING id, client_id
)
INSERT INTO rhythm_result (result_id, track_id, bpm, score, grade,
  perfect, great, good, miss, notes, max_combo)
SELECT ins.id, d.track, d.bpm, d.score, d.grade, d.perfect, d.great, d.good,
  d.miss, d.perfect + d.great + d.good + d.miss, d."maxStreak"
FROM ins JOIN data d ON d.id = ins.client_id
WHERE d.mode = 'rhythm' AND d.track IS NOT NULL;
```

> Resultados do Ritmo feitos na 1.1.0 já trazem `track`; não há resultados do Ritmo sem faixa. Se uma faixa não existir no banco, a transação falha pela chave estrangeira e nada é gravado: rode antes a carga do catálogo.

Segurança e privacidade

- **Row Level Security** em `user_preference`, `result`, `rhythm_result` e `missed_key`: cada sessão define `app.user_id` e só vê as próprias linhas; o ranking usa uma visão com só nome, pontos, nota e data.
- **Dados mínimos:** e-mail, nome de exibição e o hash da senha. Texto customizado, documentos enviados e o nome dos arquivos ficam só no navegador.
- **LGPD:** exportação em CSV pelo próprio Pluma, exclusão da conta com cascata e retenção de no máximo 5 mil resultados por pessoa (os mais antigos viram agregados mensais).
- **API:** parâmetros sempre ligados (`$1`, `$2`), limite de 500 resultados e 2 MB por sincronização, limite de requisições por conta e por IP, e validação dos mesmos intervalos do navegador antes do banco.
- **Backups:** `pg_dump` diário cifrado, retenção de 30 dias e teste de restauração mensal; o catálogo e o acervo podem ser recarregados do repositório.
- **Papéis:** um papel só de leitura para relatórios, um para a API (sem `DROP` nem `ALTER`) e o dono das migrações, usado só no deploy.

```
ALTER TABLE result ENABLE ROW LEVEL SECURITY;
CREATE POLICY result_owner ON result
  USING (user_id = current_setting('app.user_id')::uuid)
  WITH CHECK (user_id = current_setting('app.user_id')::uuid);
```

Preparar o PostgreSQL

Script completo para um banco vazio no PostgreSQL 16. Depois dele, carregue `phrase`, `word` e `track` a partir dos arquivos de `web/data` (por exemplo, com `jsonb_to_recordset`, como na migração).

```
CREATE EXTENSION IF NOT EXISTS citext;

CREATE TABLE app_user (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email         citext NOT NULL UNIQUE,
  display_name  varchar(60) NOT NULL CHECK (char_length(display_name) BETWEEN 2 AND 60),
  password_hash text NOT NULL,
  created_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz
);

CREATE TABLE track (
  id          varchar(60) PRIMARY KEY CONSTRAINT track_id_format CHECK (id ~ '^[a-z0-9-]{2,60}$'),
  title       varchar(80) NOT NULL UNIQUE,
  artist      varchar(60) NOT NULL,
  groove      varchar(12) NOT NULL CHECK (groove IN
('chip','synthwave','rock','punk','pop','disco','house','funk','samba','bossa','baiao','reggae','lofi','jazz','dnb','metal'
  bpm         smallint NOT NULL CHECK (bpm BETWEEN 60 AND 200),
  music_key   varchar(2) NOT NULL CHECK (music_key IN
('C','C#','Db','D','D#','Eb','E','F','F#','Gb','G','G#','Ab','A','A#','Bb','B')),
  scale       varchar(10) NOT NULL CHECK (scale IN
('major','minor','dorian','mixolydian','lydian','phrygian')),
  verse       smallint[] NOT NULL,
  chorus      smallint[] NOT NULL,
  lead        varchar(8) NOT NULL CHECK (lead IN ('pulse12','pulse25','square','triangle','saw')),
  arp         varchar(6) NOT NULL CHECK (arp IN ('none','up','down','updown')),
  swing       numeric(3,2) NOT NULL DEFAULT 0 CONSTRAINT track_swing_valid CHECK (swing BETWEEN 0 AND 0.4),
  active      boolean NOT NULL DEFAULT true,
  CONSTRAINT track_degrees_valid CHECK (
    cardinality(verse) BETWEEN 1 AND 16 AND cardinality(chorus) BETWEEN 1 AND 16
    AND 0 <= ALL(verse) AND 6 >= ALL(verse) AND 0 <= ALL(chorus) AND 6 >= ALL(chorus))
);

CREATE TABLE phrase (
  id         varchar(20) PRIMARY KEY,
  size       varchar(6) NOT NULL CHECK (size IN ('short','medium','long')),
  complexity varchar(6) NOT NULL CHECK (complexity IN ('easy','medium','hard')),
  body       text NOT NULL UNIQUE CHECK (char_length(body) BETWEEN 10 AND 600)
);

CREATE TABLE word (
  id    serial PRIMARY KEY,
  level varchar(6) NOT NULL CHECK (level IN ('easy','medium','hard','sounds')),
  value varchar(40) NOT NULL,
  UNIQUE (level, value)
);

CREATE TABLE user_preference (
  user_id          uuid PRIMARY KEY REFERENCES app_user(id) ON DELETE CASCADE,
  mode             varchar(6) NOT NULL DEFAULT 'test' CHECK (mode IN ('free','test','rhythm')),
  kind             varchar(10) NOT NULL DEFAULT 'text' CHECK (kind IN ('text','words','no-
accents','alphabet','numbers','numpad','data','mixed','custom')),
  size             varchar(6) NOT NULL DEFAULT 'short' CHECK (size IN ('short','medium','long')),
  complexity       varchar(6) NOT NULL DEFAULT 'easy' CHECK (complexity IN ('easy','medium','hard')),
  theme            varchar(5) CHECK (theme IN ('light','dark')),
  palette          varchar(5) NOT NULL DEFAULT 'pluma' CHECK (palette IN ('pluma','bento','sonda','nexo')),
  caret            varchar(9) NOT NULL DEFAULT 'line' CHECK (caret IN ('line','block','underline')),
  font_size        varchar(6) NOT NULL DEFAULT 'medium' CHECK (font_size IN ('small','medium','large')),
  sound            boolean NOT NULL DEFAULT false,
  keyboard         boolean NOT NULL DEFAULT true,
  effects          boolean NOT NULL DEFAULT true,
  stop_on_error    boolean NOT NULL DEFAULT false,
  music_volume     smallint NOT NULL DEFAULT 70 CHECK (music_volume BETWEEN 0 AND 100),
  rhythm_offset_ms smallint NOT NULL DEFAULT 0 CHECK (rhythm_offset_ms BETWEEN -200 AND 200),
  rhythm_track     varchar(60) REFERENCES track(id) ON DELETE SET NULL,
  updated_at       timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE TABLE result (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
  client_id       varchar(24) NOT NULL,
  played_at       timestamptz NOT NULL,
  mode            varchar(6) NOT NULL CHECK (mode IN ('free','test','rhythm')),
  kind            varchar(10) NOT NULL CHECK (kind IN ('text','words','no-
accents','alphabet','numbers','numpad','data','mixed','custom')),
  size            varchar(6) NOT NULL CHECK (size IN ('short','medium','long')),
  complexity      varchar(6) NOT NULL CHECK (complexity IN ('easy','medium','hard')),
  category        text GENERATED ALWAYS AS (
                    CASE WHEN kind = 'custom' THEN 'custom' ELSE kind || '/' || size || '/' || complexity
END) STORED,
  seed            bigint CHECK (seed BETWEEN 0 AND 4294967295),
  phrase_id       varchar(20) REFERENCES phrase(id) ON DELETE SET NULL,
  texts           integer NOT NULL DEFAULT 1 CHECK (texts >= 1),
  chars           integer NOT NULL CHECK (chars >= 0),
  duration_ms     integer NOT NULL CHECK (duration_ms >= 0),
  keystrokes      integer NOT NULL CHECK (keystrokes >= 0),
  correct         integer NOT NULL CHECK (correct >= 0),
  errors          integer NOT NULL CHECK (errors >= 0),
  uncorrected     integer NOT NULL CHECK (uncorrected >= 0),
  accuracy_bp     integer NOT NULL CHECK (accuracy_bp BETWEEN 0 AND 10000),
  net             integer NOT NULL CHECK (net >= 0),
  net_per_min_x100 integer NOT NULL CHECK (net_per_min_x100 >= 0),
  wpm_x100        integer NOT NULL CHECK (wpm_x100 >= 0),
  consistency_bp  integer CHECK (consistency_bp BETWEEN 0 AND 10000),
  max_streak      integer NOT NULL CHECK (max_streak >= 0),
  created_at      timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT result_client_unique UNIQUE (user_id, client_id),
  CONSTRAINT result_correct_valid CHECK (correct <= keystrokes),
  CONSTRAINT result_uncorrected_valid CHECK (uncorrected <= errors)
);

CREATE TABLE rhythm_result (
  result_id   uuid PRIMARY KEY REFERENCES result(id) ON DELETE CASCADE,
  track_id    varchar(60) NOT NULL REFERENCES track(id),
  bpm         smallint NOT NULL CHECK (bpm BETWEEN 60 AND 200),
  score       integer NOT NULL CHECK (score >= 0),
  grade       char(1) NOT NULL CONSTRAINT rhythm_grade_valid CHECK (grade IN ('S','A','B','C','D')),
  perfect     integer NOT NULL CHECK (perfect >= 0),
  great       integer NOT NULL CHECK (great >= 0),
  good        integer NOT NULL CHECK (good >= 0),
  miss        integer NOT NULL CHECK (miss >= 0),
  notes       integer NOT NULL,
  wrong_keys  integer NOT NULL DEFAULT 0 CHECK (wrong_keys >= 0),
  max_combo   integer NOT NULL DEFAULT 0 CHECK (max_combo >= 0),
  holds_full  integer NOT NULL DEFAULT 0 CHECK (holds_full >= 0),
  holds_total integer NOT NULL DEFAULT 0,
  offset_ms   smallint CHECK (offset_ms BETWEEN -140 AND 140),
  CONSTRAINT rhythm_notes_sum CHECK (perfect + great + good + miss = notes),
  CONSTRAINT rhythm_holds_valid CHECK (holds_full <= holds_total OR holds_total = 0)
);

CREATE TABLE missed_key (
  result_id uuid NOT NULL REFERENCES result(id) ON DELETE CASCADE,
  key_char  varchar(4) NOT NULL,
  count     integer NOT NULL CHECK (count > 0),
  PRIMARY KEY (result_id, key_char)
);

CREATE INDEX result_user_played ON result (user_id, played_at DESC);
CREATE INDEX result_records ON result (user_id, mode, category, net_per_min_x100 DESC);
CREATE INDEX result_tests ON result (user_id, played_at DESC) WHERE mode = 'test';
CREATE INDEX rhythm_ranking ON rhythm_result (track_id, score DESC);
CREATE INDEX missed_key_char ON missed_key (key_char);

CREATE VIEW typing_record AS
SELECT DISTINCT ON (user_id, category) user_id, category, id AS result_id,
```

```
       net_per_min_x100, accuracy_bp, played_at
FROM result
WHERE mode = 'test'
ORDER BY user_id, category, net_per_min_x100 DESC, played_at;

CREATE VIEW rhythm_record AS
SELECT DISTINCT ON (r.user_id, rr.track_id, r.category)
       r.user_id, rr.track_id, r.category, rr.result_id, rr.score, rr.grade, r.played_at
FROM result r
JOIN rhythm_result rr ON rr.result_id = r.id
ORDER BY r.user_id, rr.track_id, r.category, rr.score DESC, r.played_at;

ALTER TABLE user_preference ENABLE ROW LEVEL SECURITY;
ALTER TABLE result ENABLE ROW LEVEL SECURITY;
ALTER TABLE rhythm_result ENABLE ROW LEVEL SECURITY;
ALTER TABLE missed_key ENABLE ROW LEVEL SECURITY;

CREATE POLICY preference_owner ON user_preference
  USING (user_id = current_setting('app.user_id')::uuid)
  WITH CHECK (user_id = current_setting('app.user_id')::uuid);
CREATE POLICY result_owner ON result
  USING (user_id = current_setting('app.user_id')::uuid)
  WITH CHECK (user_id = current_setting('app.user_id')::uuid);
CREATE POLICY rhythm_owner ON rhythm_result
  USING (EXISTS (SELECT 1 FROM result r WHERE r.id = result_id
                 AND r.user_id = current_setting('app.user_id')::uuid));
CREATE POLICY missed_owner ON missed_key
  USING (EXISTS (SELECT 1 FROM result r WHERE r.id = result_id
                 AND r.user_id = current_setting('app.user_id')::uuid));
```